Homework 2

Practicing Python: strings, lists, and dictionaries

You will accept the homework invitation through the GitHub Classroom link in Canvas. This creates a `hw2-python-yourname` repository. Clone it and put each script in the top level of that repository.

For every problem, write the functions described below and a short `if __name__ == "__main__":` block that runs them on the example data and prints the results. Your scripts should run with `python3 problemN.py` from the repository directory, and every function should have a one-line docstring.

Do not use `pandas`, `Biopython`, or `re` in this assignment. The goal is to practice loops, lists, strings, and dictionaries.

Problem 1: Iterating on a string (`problem1.py`)

A DNA sequence is a string. In this problem you will loop over its characters.

a. Write `count_bases(seq)`. It loops over every character in `seq` and returns a dictionary with the counts of `A`, `C`, `G`, and `T`.

```
>>> count_bases("ATGGCGTACGTTAGC")
{'A': 3, 'C': 3, 'G': 5, 'T': 4}
```

Make the function work for lower-case input too (`"atgg"` should count the same as `"ATGG"`).

b. Write `reverse_complement(seq)`. It returns the reverse complement of the sequence. A -> T, T -> A, G -> C, C -> G, read from the end of the string toward the start.

```
>>> reverse_complement("ATGGCGTACGTTAGC")
'GCTAACGTACGCCAT'
```

c. Write `find_motif_positions(seq, motif)`. It returns a list of **1-based** start positions where `motif` occurs in `seq`. Count overlapping matches. Use a loop over the start positions, not `str.find` or `str.count`.

```
>>> find_motif_positions("ATATATA", "ATA")
[1, 3, 5]
>>> find_motif_positions("ATGCCC", "TTT")
[]
```

d. Write `gc_windows(seq, window)`. For every window of length `window` (sliding one base at a time), return the fraction of bases that are G or C. Return a list of floats with one value per window.

```
>>> gc_windows("GGCCAATT", 4)
[1.0, 0.75, 0.5, 0.25, 0.0]
```

Bonus: Write `split_codons(seq)`. It returns a list of consecutive 3-letter codons. Ignore any incomplete codon at the end.

```
>>> split_codons("ATGGCGTACG")
['ATG', 'GCG', 'TAC']
```

Problem 2: Iterating on a list, finding and counting (`problem2.py`)

The file `gene_lengths.txt` contains one gene length (in bp) per line. Create this file in your repository with these eight values:

```
1100
750
1800
```

```
950
1500
620
2400
880
```

Your script reads the file into a list of integers (write `read_lengths(filename)` for this). Then write the following functions.

a. `count_above(values, cutoff)`. Returns how many values are strictly greater than `cutoff`.

```
>>> count_above(lengths, 1000)
4
```

Problem 3: Merging two datasets with a dictionary (`problem3.py`)

Two tab-separated files describe the same set of genes. Create them in your repository, using real tab characters between columns.

`annotations.tsv`

```
gene_id chrom   product
g001    chr1    DNA polymerase
g002    chr1    ribosomal protein L2
g003    chr2    ATP synthase subunit a
g004    chr2    hypothetical protein
g005    chr3    transposase
```

`expression.tsv`

```
gene_id tpm
g001    35.2
g002    110.8
g003    4.1
g005    0.0
g006    58.9
```

Both files have a header line. Skip it when you read the data.

a. Write `read_annotations(filename)`. It returns a dictionary keyed by `gene_id`. Each value is a tuple `(chrom, product)`.

b. Write `read_expression(filename)`. It returns a dictionary keyed by `gene_id`. Each value is the TPM as a `float`.

c. Write `merge_datasets(annotations, expression)`. It returns a dictionary keyed by `gene_id`, containing only genes present in **both** inputs. Each value is a tuple `(chrom, product, tpm)`.

d. Write `write_merged(merged, filename)`. It writes `merged.tsv` with a header line (`gene_id`, `chrom`, `product`, `tpm`) and one row per gene, sorted by `gene_id`. Your output file should contain:

```
gene_id chrom   product tpm
g001    chr1    DNA polymerase  35.2
g002    chr1    ribosomal protein L2    110.8
g003    chr2    ATP synthase subunit a  4.1
g005    chr3    transposase 0.0
```

e. Print to STDOUT the genes that are in only one of the two files:

```
Only in annotations: g004
Only in expression: g006
```

f. Using the merged dictionary, count the number of genes on each chromosome, and print the counts:

```
chr1 2
chr2 1
chr3 1
```

g. For each chromosome, compute the mean TPM of genes with **TPM greater than 0**. Print the chromosome and mean, rounded to one decimal place. Skip chromosomes that have no genes with TPM > 0.

```
chr1 73.0
chr2 4.1
```

(`chr3` is not printed because its only gene has TPM 0.0.)

Submission

Commit and push to your repository:

- `problem1.py`, `problem2.py`, `problem3.py`
- `gene_lengths.txt`, `annotations.tsv`, `expression.tsv`
- `merged.tsv` (generated by `problem3.py`)

Your scripts should give the example outputs shown above when run on the data files as written. Check your work on the examples before you commit. The bonus parts are optional.